

Formation FPGA

TP3 : FSM

1. Introduction

L'objectif de ce troisième TP est la modélisation et la réalisation d'une application mettant en œuvre une machine d'états sous l'environnement Vivado. La manipulation consiste en la description matérielle du circuit, incluant le compteur développé précédemment et une unité de contrôle pour la FSM. Elle est suivie de la validation de sa logique par simulation fonctionnelle (Testbench) et de l'analyse de ses contraintes temporelles. Enfin, l'implémentation et la programmation du Bitstream sur la carte FPGA Cora Z7 permettront la démonstration physique du système par le contrôle de tous les cycles de la machine d'états associées aux sorties LEDs.

2. Questions

2.1. Question 1

```
entity counter_unit is
  generic(
    constant VAL_MAX : integer := 199999999 --par défaut : 2
  );
  port (
    clk      : in std_logic;
    resetn   : in std_logic;
    end_counter : out std_logic
  );
end counter_unit;

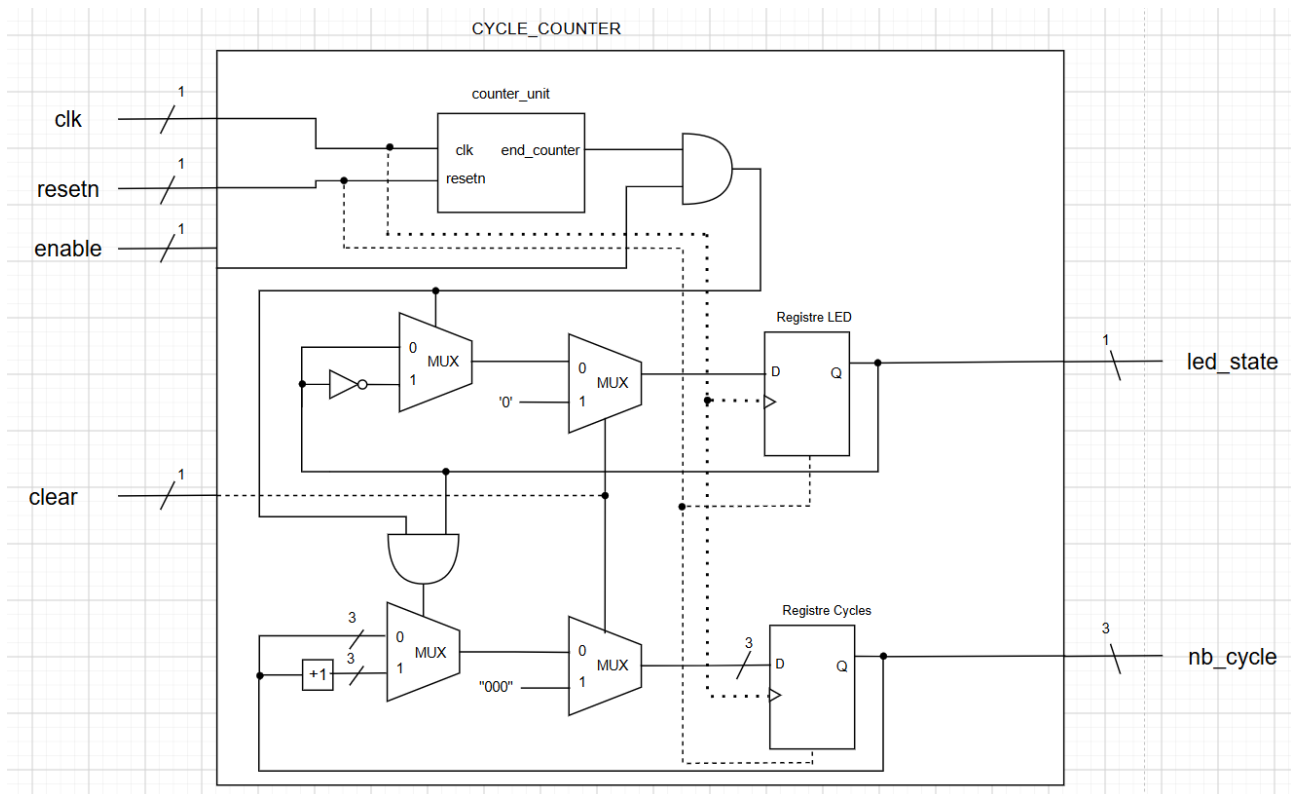
architecture behavioral of counter_unit is
  -- Declaration des signaux internes
  signal cpt_reg : unsigned (27 downto 0) := (others => '0') ;

  begin
    --Partie séquentielle
    process(clk, resetn)
    begin
      if(resetn = '0') then
        cpt_reg <= (others => '0');
      elsif(rising_edge(clk)) then
        if(cpt_reg = VAL_MAX) then
          cpt_reg <= (others => '0');
        else
          cpt_reg <= cpt_reg + 1;
        end if;
      end if;
    end process;

    --Partie combinatoire
    end_counter <= '1' when (cpt_reg = VAL_MAX)
      else '0';
  end behavioral;
```

L'entité « counter_unit » sera utilisé tel un module pour les applications suivantes durant ce TP. Il s'agit d'un simple compteur mais la spécificité est l'argument generic() qui permet à un utilisateur de pouvoir modifier la valeur ou seuil maximal à sa guise si le module est utilisé comme un « component » pour un autre bloc logique. La durée de clignotement d'une LED en est un exemple. Cette instruction supplémentaire permet de ne plus modifier ce fichier .vhd pour éviter la modification de la valeur maximale.

2.2. Question 2



Le schéma RTL théorique ci-dessus représente l'architecture globale de notre compteur de cycles utilisant ainsi le module « **counter_unit** ». Plusieurs entrées et sorties répondant au besoin de l'application ont dû être ajoutées ainsi que les logiques séquentielles et combinatoires qui vont avec:

- Les entrées « clear » - « enable » : Le compteur doit pouvoir se remettre à 0 ou bien s'incrémenter selon le souhait de l'utilisateur à n'importe quel moment.
- La sortie « led_state » : La LED doit clignoter une fois que le seuil maximal du simple compteur « **counter_unit** » atteint. Un signal interne sera utilisé tel un flag lié au signal « end_counter » autorisant le clignotement. Un registre est utilisé afin de mémoriser le dernier état de la LED.
- La sortie « nb_cycle » : Un cycle correspond à une LED éteinte puis allumée. Cette variable aura comme valeur le nombre de cycles comptés.

La logique combinatoire avec les multiplexeurs analyse en permanence les entrées de contrôle et l'état actuel des registres afin de calculer instantanément leurs futures valeurs.

- La gestion de la LED s'appuie sur un registre et précédé d'une cascade de multiplexeurs. le premier multiplexeur prend en entrée la rétroaction de la sortie du registre afin de garder la même valeur ou inverser la valeur de la LED, un signal de commande équivalent au résultat

entre « enable » **AND** « end_counter » pour la fin du comptage afin de vérifier l'entrée à sélectionner.

- La gestion du compteur de cycles fonctionne pratiquement sur le même principe mais seulement avec une particularité en plus, afin d'incrémenter le compteur, la valeur du registre LED doit être connu afin de d'identifier la fin d'un cycle ON/OFF, justifiant ainsi l'utilisation d'une autre porte **AND**.
- L'entrée « clear » est utilisé comme signal de commande pour les MUX suivants afin de forcer la remise à zéro si nécessaire, donnant ainsi la priorité à cette fonctionnalité. Placer ces multiplexeurs au plus proche des registres insistent sur cette priorité et de ne prendre en compte la sortie précédente si une remise à zéro est demandée.

2.3. Question 3

```
entity cycle_counter is
  generic (
    -- Par défaut : 1 seconde d'attente a 100 MHz (clignotement total sur 2s)
    TOGGLE_DELAY : integer := 100000000
  );
  port (
    clk      : in  std_logic;
    resetn   : in  std_logic;
    enable   : in  std_logic; -- '1' pour autoriser le comptage
    clear    : in  std_logic; -- '1' pour forcer la remise a 0
    led_state : out std_logic; -- Sortie pour faire clignoter la LED (1s On / 1s Off)
    nb_cycle : out std_logic_vector(2 downto 0) -- Valeur brute du compteur pour le no
  );
end entity;

architecture Behavioral of cycle_counter is

  -- Déclaration du composant counter_unit
  component counter_unit is
    generic (
      VAL_MAX : integer
    );
    port (
      clk      : in  std_logic;
      resetn   : in  std_logic;
      end_counter : out std_logic
    );
  end component;

  -- Signaux internes
  signal finish_flag : std_logic := '0';
  signal led_reg     : std_logic := '0';
  signal cpt_reg     : unsigned(3 downto 0) := "0000";

begin

  -- Instanciation du module counter_unit
  counter : counter_unit
    generic map (
      VAL_MAX => TOGGLE_DELAY -- Prend la valeur 125 000 000 par défaut
    )
    port map (
      clk      => clk,
      resetn   => resetn,
      end_counter => finish_flag
    );

  -- Process de gestion du compteur et du clignotement
  process(clk, resetn)
  begin
    if resetn = '0' then
      cpt_reg <= (others => '0');
      led_reg  <= '0';
    elsif rising_edge(clk) then

      if clear = '1' then
        cpt_reg <= (others => '0'); -- Remise à 0
        led_reg  <= '0';

      elsif enable = '1' and finish_flag = '1' then
        --cpt_reg <= cpt_reg + 1; -- +1 au compteur de changements
        led_reg <= not led_reg; -- Inversion de l'état (Toggle toutes les 1s)

        -- Ne s'incrémente que si la LED était à '1' et qu'elle s'éteint
        -- Cela valide un cycle complet (Allumé puis Éteint)
        if led_reg = '1' then
          cpt_reg <= cpt_reg + 1;
        end if;
      end if;
    end if;
  end process;

  -- Affectation des sorties
  led_state <= led_reg;
  nb_cycle <= std_logic_vector(cpt_reg);

end Behavioral;
```

Le code VHDL représenté décrit matériellement notre compteur de cycles utilisant le module développé précédemment « **counter_unit** ». La sortie « *led_state* » : La LED doit clignoter une fois que le seuil maximal (défini en tant que constante *generic(Q)*) du simple compteur « **counter_unit** » atteint. Un signal interne « *finish_flag* », comme son nom l'indique, est utilisé tel un flag lié au signal « *end_counter* » autorisant le clignotement. Un registre est utilisé afin de mémoriser le dernier état de la LED.

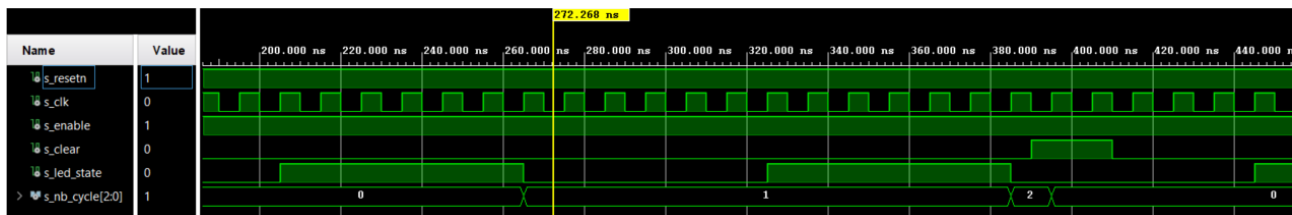
Un cycle correspond à une LED éteinte puis allumée, d'où son incrémentation uniquement quand le dernier état de la LED connu est « 1 », c'est à dire Allumée. Cette sortie est donc affectée à un registre interne qui se charge de compter le nombre de cycles au cours de son utilisation.

La création d'une entité indépendante permettra donc de pouvoir l'utiliser pour les prochaines applications sans avoir à le modifier et permettant à un utilisateur d'insérer directement le temps de clignotement souhaité.

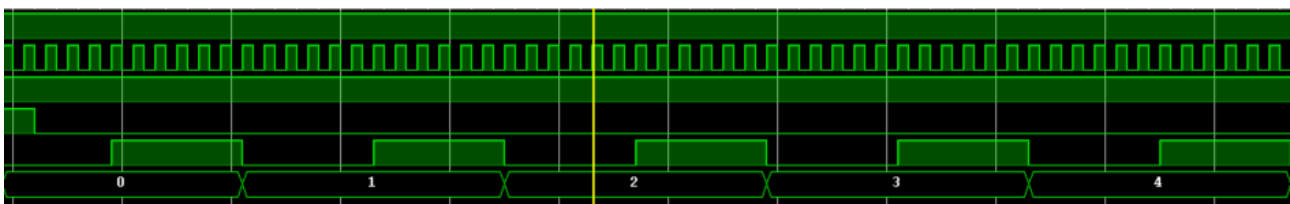
La durée de changement d'état pour passer de Off à On et inversement pour la LED est de 1 seconde (fréquence d'horloge est à 125 MHz donc la valeur peut être différente de celle dans le code).

2.4. Question 4

Afin de tester le bon fonctionnement de notre compteur de cycles, deux paramètres seront observés : la fonction de remise à zéro ainsi que le comptage de cycles.



On définit un délai de clignotement sur 5 cycles d'horloge. Le signal « *clear* » est à l'état haut au bout de 2 cycles de comptage puis remis à zéro au prochain front montant d'horloge, validant ainsi son bon fonctionnement.

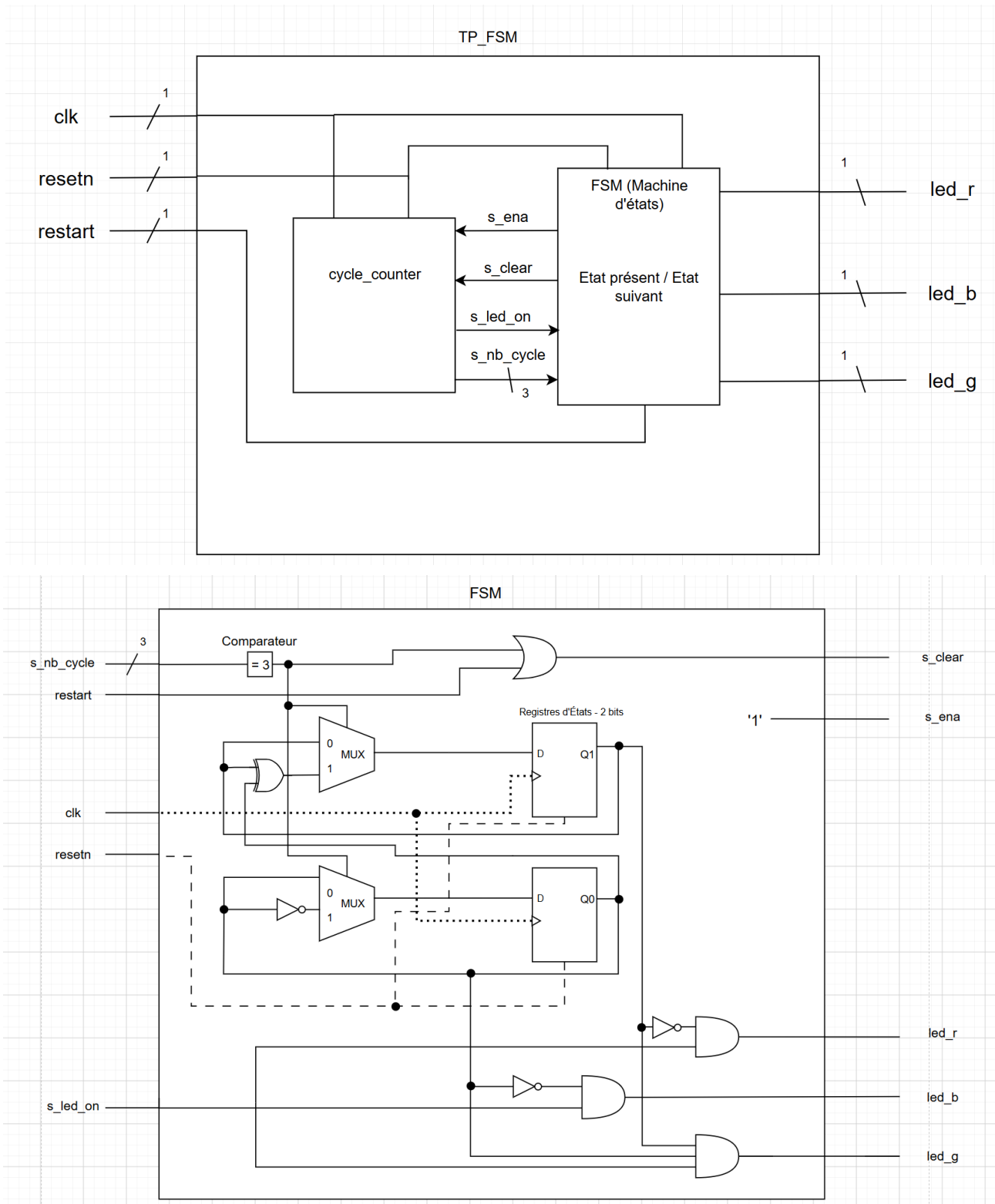


Le chronogramme suivant montre le clignotement de la LED sur le délai fixé et l'incrémentation du compteur uniquement effectué sur le cycle On/Off validant ainsi ce paramètre également. Le fonctionnement global en simulation est donc satisfaisant.

Quelques tests assert et des commentaires ont également été ajoutés au testbench, avec un test négatif afin de forcer l'affichage dans la console :

```
# run 1000ns
Error: ! Erreur : Le compteur de cycles ne vaut pas 2
```

2.5. Question 5



L'architecture globale de cette entité correspond à une combinaison entre unité de traitement et unité de commande.

Les signaux d'entrée et sortie du module « **cycle_counter** » se transforment en signaux internes et cette communication entre ces deux blocs vont permettre de créer une machine d'états avec quatre états distincts et des transitions pour chacune.

Typiquement, l'unité de contrôle décide du comptage et de sa remise à zéro, tandis que le compteur lui transmettra le nombre de cycles atteint afin de gérer la transition avec l'état suivant et d'envoyer le signal électrique nécessaire au bon affichage des couleurs de la LED.

Les broches en sortie sont liées à une couleur chacune représentant la LED RGB qui seront définies dans le fichier de contraintes.

Pour aller plus en profondeur, le bloc FSM présente plusieurs spécificités :

- Le registre 2 bits pour la machine d'états nécessite deux bascules avec donc 2 sorties.
 - ▶ « 00 » pour l'état IDLE, « 01 » pour l'état Rouge, « 10 » pour l'état Bleu et « 11 » pour l'état Vert
 - ▶ 2 multiplexeurs sont donc nécessaires afin de gérer la transition état présent et état suivant et de changer les valeurs en entrée des registres selon la valeur du signal de commande en sortie du comparateur (avec le signal « s_nb_cycle »).
 - ▶ Prenons typiquement un exemple, État_present = Rouge ($Q_1 = 0$, $Q_0 = 1$), on veut obtenir la valeur $Q_1 = 1$, $Q_0 = 0$ pour État_suivant = Bleu :

Avec « s_nb_cycle » = 3, les Mux sélectionnent les entrées 1

→ Mux 1 = $Q_1 \text{ XOR } Q_0 = 0 \text{ XOR } 1 = 1$ (Entrées différentes)

→ Mux 0 = $\text{NOT } Q_0 = 0$ - Le circuit passera à l'état Bleu pour le prochain front d'horloge

- ▶ Pour la LED RGB, il suffit seulement d'associer aux sorties le signal de clignotement « s_led_on » provenant du compteur de cycles et de prendre les sorties des registres correspondantes :

→ La LED rouge ne clignote que durant les états IDLE et Rouge donc $Q_1 = 0$ est toujours valable – $\text{NOT } Q_1$ en entrée de la porte **AND**

→ La LED bleue ne clignote que durant les états IDLE et Bleu donc $Q_0 = 0$ est toujours valable - $\text{NOT } Q_1$ en entrée de la porte **AND**

→ La LED verte ne clignote que durant les états IDLE et Vert donc $Q_0 = Q_1$ – Les deux sorties sont associées à la porte **AND** à 3 entrées

2.6. Question 6

Entrées	Sorties	Signaux internes
Clk (Horloge)	led_r (Broche LED – Rouge)	s_ena (Entrée Enable – Compteur)
Resetn (Reset inversé)	led_b (Broche LED - Bleu)	s_clear (Entrée Clear - Compteur)
Restart (Remise à 0 - Bouton)	led_g (Broche LED - Vert)	s_led_on (Sortie led_state - Compteur)
		s_nb_cycle (Sortie nb_cycles - Compteur)
		etat_present (Machine d'états)
		etat_suivant (Machine d'états)

2.7. Question 7

```

entity tp_fsm is
  generic(
    TOGGLE_DELAY : integer := 100000000 --Horloge à 100 MHz -> 1 seconde ;
  );
  port (
    clk      : in std_logic;
    resetn   : in std_logic;
    restart  : in std_logic;
    led_r    : out std_logic;
    led_b    : out std_logic;
    led_g    : out std_logic -- Le bus 3 bits pour les pins
  );
end tp_fsm;

architecture behavioral of tp_fsm is

  component cycle_counter is
    generic (
      TOGGLE_DELAY : integer
    );
    port (
      clk      : in std_logic;
      resetn   : in std_logic;
      enable   : in std_logic;
      clear    : in std_logic;
      led_state : out std_logic;
      nb_cycle : out std_logic_vector(2 downto 0)
    );
  end component;

  type state is (idle, red, blue, green); --a modifier avec vos etats

  signal current_state : state; --etat dans lequel on se trouve actuellement
  signal next_state : state; --etat dans lequel on passera au prochain cot

  signal s_ena, s_clear, s_led_on : std_logic; -- Signaux internes pour le
  signal s_nb_cycle : std_logic_vector(2 downto 0);

begin

  s_ena <= '1';
  Counter : cycle_counter
    generic map (
      TOGGLE_DELAY => TOGGLE_DELAY
    )
    port map (
      clk      => clk,
      resetn   => resetn,

```

```

enable      => s_ena,
clear       => s_clear,
led_state   => s_led_on,
nb_cycle    => s_nb_cycle
);

Control_Unit : process(clk, resetn, restart)
begin
    if(resetn = '0' or restart = '1' ) then

        current_state <= idle;

    elsif(rising_edge(clk)) then

        current_state <= next_state;

    end if;
end process Control_Unit;

-- FSM
State_Machine : process(current_state, s_nb_cycle)
begin
    case current_state is -- Définition des transis:
        when idle =>
            if(s_nb_cycle = 3) then
                next_state <= red;
            else
                next_state <= idle;
            end if;

        when red =>
            if(s_nb_cycle = 3) then
                next_state <= blue;
            else
                next_state <= red;
            end if;

        when blue =>
            if(s_nb_cycle = 3) then
                next_state <= green;
            else
                next_state <= blue;
            end if;

        when green =>
            if(s_nb_cycle = 3) then
                next_state <= idle;
            else
                next_state <= green;
            end if;
    end case;

end process State_Machine;

--Logique combinatoire
s_clear <= '1' when (s_nb_cycle = 3 or restart = '1') else '0'; --Remise directe à zéro s

--Affectation des sorties
Outputs : process(current_state, s_led_on)
begin
    case current_state is -- Utilisation de la sortie du compteur de cycles
        when idle => --Mise à au niveau bas des couleurs non utilisées
            led_r <= s_led_on; led_b <= s_led_on; led_g <= s_led_on;

        when red =>
            led_r <= s_led_on; led_b <= '0'; led_g <= '0';

        when blue =>
            led_r <= '0' ; led_b <= s_led_on; led_g <= '0';

        when green =>
            led_r<='0'; led_b <= '0'; led_g <= s_led_on;

        when others =>
            led_r <= '0'; led_b <= '0'; led_g <= '0';

    end case;
end process Outputs;

end behavioral;

```


Ce code VHDL retranscrit le schéma RTL théorique présenté précédemment. Il est défini en trois process principaux :

- Un premier process, l'unité de contrôle, qui synchronise la machine d'états avec l'horloge et se remettant à l'état repos en cas de contact avec le reset hardware ou le bouton restart.
- Un deuxième process, **State_Machine**, qui se charge de la transition entre chaque état l'aide d'un switch case (beaucoup plus optimal et lisible que de simples if/else). Les variables « current_state » et s_nb_cycle » sont ajoutés dans la liste de sensibilité car la machine d'états se fient à eux afin d'assurer le passage à un état suivant si le nombre de cycles max est atteint.
- Le troisième process permet de gérer convenablement les états de sortie, donc l'affichage de la LED RGB, en prenant en compte l'état présent ainsi le signal de l'état de clignotement actuel (ON ou OFF) selon la durée définie dans la section **generic()**.

L'ajout d'une constante générique pour cette entité également laisse l'utilisateur définir la durée de clignotement souhaité. La valeur est donc associée à l'argument générique du module « **cycle_counter** » lors de son instantiation dans le programme.

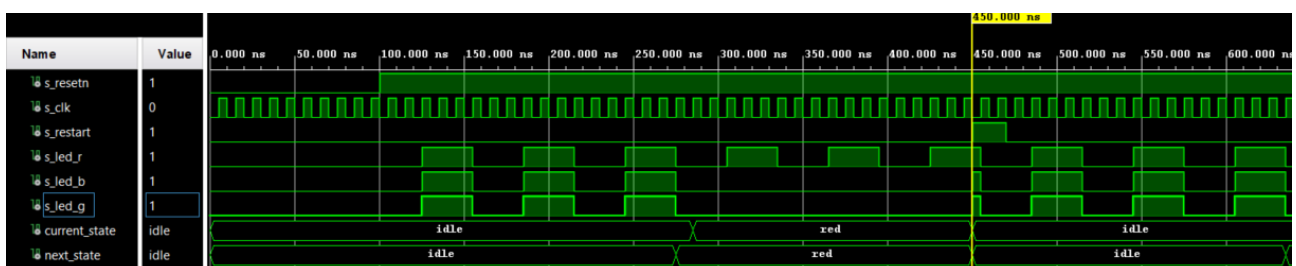
Le signal « s_ena » relié à « enable » est fixé à 1 dès le début. Ce choix se justifie par l'absence d'une spécification pour ce signal. Le compteur comptera automatiquement et synchronisé avec l'horloge et déroulant ainsi les différents états.

Bien que la logique de remise à zéro du compteur (« s_clear ») soit gérée de manière purement combinatoire en fonction de la valeur limite « s_nb_cycle = 3 », les temps de propagation permettent à la machine d'états de capturer la transition et de changer d'état au front montant d'horloge avant l'effacement effectif des registres du compteur. Cependant, le code peut être plus robuste en intégrant le signal « s_clear » directement dans le process séquentiel de la machine d'états pour permettre de synchroniser la remise à zéro du compteur lors d'un changement d'état.

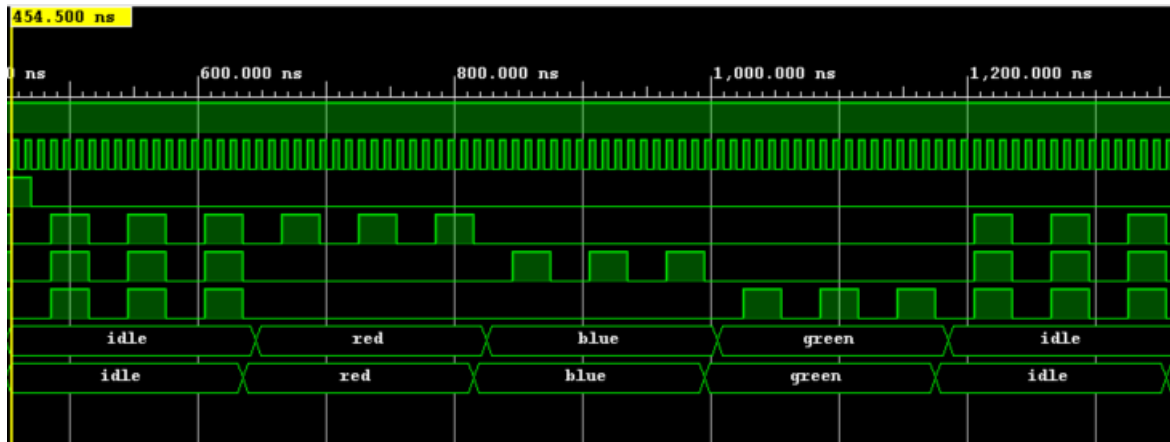
La valeur de la constante TOGGLE_DELAY peut être modifiée étant donné que la fréquence d'horloge est à 125 MHz, donc 1 seconde correspond à un autre nombre de périodes.

2.8. Question 8

Comme la précédente simulation, l'objectif est de tester le restart afin de vérifier si la machine d'états retourne bien à l'état initial. Ainsi que le bon fonctionnement de la machine d'état avec chaque transition et les 3 clignotements pour chaque couleur.



Lors d'un appui sur restart, le système repasse l'état initial et la LED au niveau bas au prochain front montant d'horloge recommençant ainsi tout le process et les différents clignotements.



Le chronogramme ci-dessus représente le fonctionnement correct de la machine d'états avec le passage par les 4 états définis pour chaque couleur : blanc, rouge, bleu et vert ainsi que leurs 3 cycles de clignotements successifs, finalisé par le retour à l'état initial. La simulation est donc validée.

```

Note: - OK - Demarrage en etat IDLE (leds blanches) valide
Time: 100 ns Iteration: 0 Process: /tb_tp_fsm/line__59 File: E:/
Note: - OK - Mode RED valide (leds Bleue et Verte bien eteintes) --
Time: 400 ns Iteration: 0 Process: /tb_tp_fsm/line__59 File: E:/
Note: - OK - Commande RESTART executee - retour a l'etat IDLE valide
Time: 471 ns Iteration: 0 Process: /tb_tp_fsm/line__59 File: E:/
Note: - SIMULATION DE LA FSM GLOBALE TERMINEE -

```

Quelques tests « assert » pour montrer ce à quoi peut par exemple ressembler une validation industrielle.

2.9. Question 9

Pour une première approche théorique, l'implémentation nécessitera 3 grandes catégories de ressources :

- Tout d'abord, les bascules **Flip Flop** :
 - Le compteur de temps aura besoin de 28 bits pour compter jusqu'à 1 seconde (le délai de clignotement)
 - Le compteur de cycles codé sur 3 bits, 3 bascules FF supplémentaires
 - Un registre pour la LED sur 1 bit simple et un registre pour la machine d'états (1 bit pour chaque état), donc 4 bascules supplémentaires.

Le total de bascules estimées seraient aux alentours de 36 bascules.

- Ensuite, les Look-Up Tables ou LUT, sont les éléments de logique combinatoire pure. Elles servent à calculer les transitions et les sorties. On peut en trouver à plusieurs entrées dont à 2, les plus utilisées par le synthétiseur.
 - ▶ Le décodage des 4 états de la FSM. (1 à 2 LUTs par état)
 - ▶ Les comparateurs pour détecter la fin du compteur de temps. (20 à 25 LUTs)
 - ▶ Les multiplexeurs pour aiguiller les signaux `led_r`, `led_g`, `led_b` en fonction de l'état.

Le total de LUTs estimés serait entre 40 à 50.

- Puis, les Buffers d'entrée et sortie pour les connexions physiques avec le monde extérieur, plus facilement visualisable :
 - ▶ Les entrées du bloc « **tp_fsm** » avec `clk`, `resetn` et `restart` donc 3 broches
 - ▶ Les sorties `led_r`, `led_b` et `led_g` donc 3 broches également.

Suite à au lancement de la synthèse, les différents composants utilisés sont repertoriés dans le tableau suivant

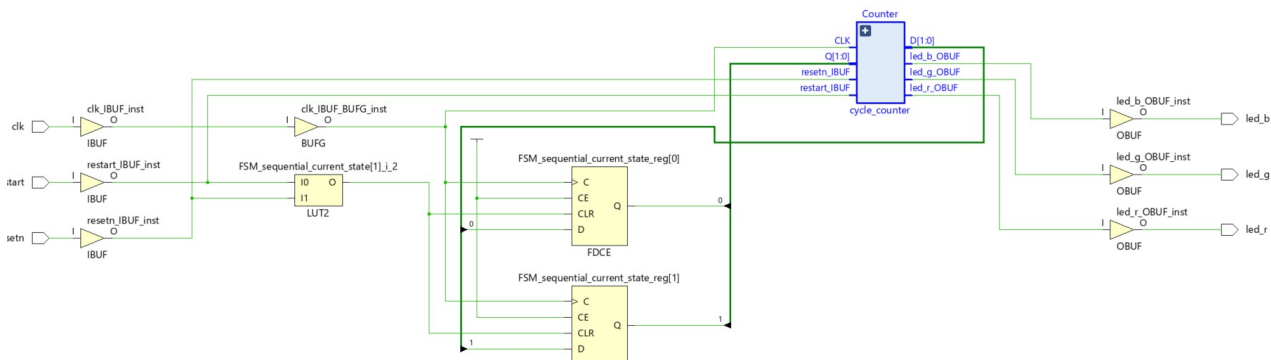
7. Primitives

Ref Name	Used	Functional Category
FDCE	33	Flop & Latch
LUT2	30	LUT
LUT4	7	LUT
CARRY4	7	CarryLogic
LUT5	6	LUT
OBUF	3	IO
IBUF	3	IO
LUT3	2	LUT
LUT1	1	LUT
BUFG	1	Clock

Le rapport de synthèse valide nos estimations théoriques. Les 33 bascules FDCE physiques correspondent aux registres requis pour la mémorisation du temps, des cycles et de l'état de la FSM. De plus, la logique combinatoire globale consomme 46 LUTs, ce qui s'inscrit plus ou moins dans nos prévisions.

L'utilisation de petites structures (comme les 30 LUT2) couplées aux 7 blocs de propagation de retenue CARRY4 démontre que l'outil de synthèse a optimisé les équations mathématiques du diviseur de temps pour minimiser l'architecture, garantissant un circuit compact, stable et économe en ressources.

Concernant le schéma de synthèse élaboré et optimisé par Vivado, le compteur de cycles se trouve ainsi sur la représentation ci-dessous :



On peut remarquer que la structure du schéma RTL synthétisé est différente de notre hiérarchie logicielle VHDL en raison des algorithmes d'optimisation de Vivado. L'outil a supprimé les frontières virtuelles de nos modules, qui sont initialement distincts dans la description matérielle, pour regrouper la logique combinatoire de contrôle des LED au plus près des signaux sources, tout en isolant les registres de la FSM, qui gèrent uniquement les différents états en fonction de l'horloge. Cette restructuration matérielle permet de minimiser le chemin critique, de réduire le nombre de LUTs nécessaires et de garantir les performances temporelles du système.

2.10. Question 10

```
# PL System Clock
set_property -dict { PACKAGE_PIN H16  IOSTANDARD LVCMOS33 } [get_ports { clk }]; #IO_L13P_T2_MRCC_35 Sch=sysclk
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { clk }];#set
```

Pour adapter notre horloge à une fréquence de 100 MHz, une instruction est modifiée dans le fichier de contraintes. La période est ainsi définie à 10 ns et le rapport cyclique de 0 à 50 % (50 % niveau haut et 50 % niveau bas) contre 8 ns comme période (horloge à 125 MHz). Cependant, la modification de cette partie n'influe en rien sur le fonctionnement de l'horloge en temps réel sur la carte en temps réel, qui reste figée à 125 MHz par le constructeur. Cette modification est surtout utile pour mesurer les performances temporelles dans les rapports de timing. La fréquence d'horloge peut être modifiée dans une IP spécifique mais cela n'est pas testé dans notre cas.

2.11. Question 11

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3,898 ns	Worst Hold Slack (WHS): 0,223 ns	Worst Pulse Width Slack (WPWS): 3,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 33	Total Number of Endpoints: 33	Total Number of Endpoints: 34

All user specified timing constraints are met.

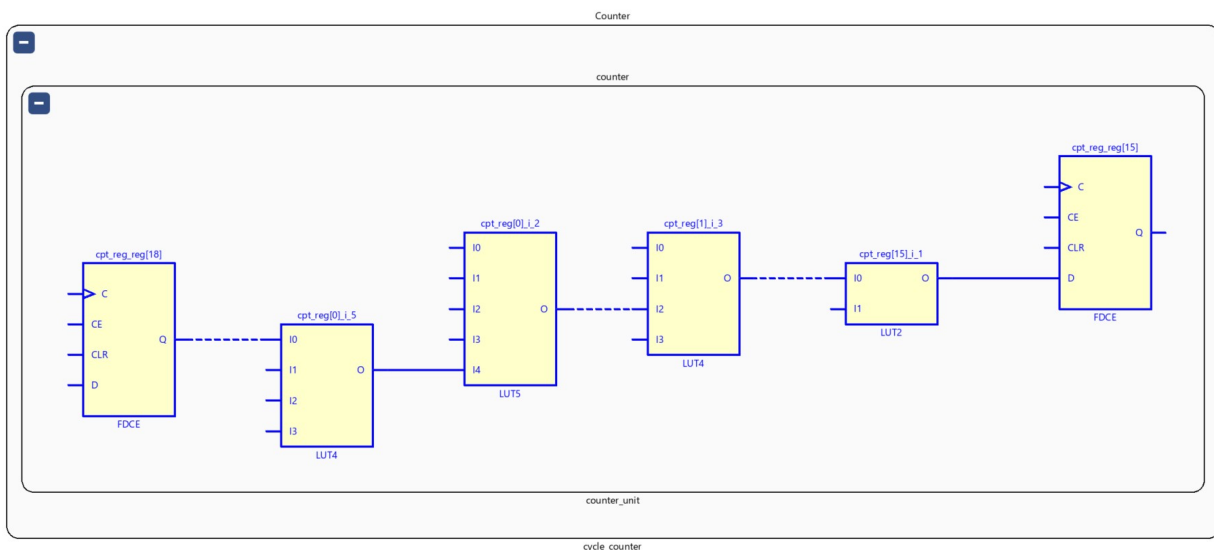
Afin d'obtenir une analyse temporelle pure de notre logique utilisateur, l'analyseur logique de debug (ILA) a été désactivé. Le rapport d'implémentation final montre un respect parfait des contraintes de timing :

- Worst Negative Slack (WNS) : Positif (pas de violation de Setup).
- Worst Hold Slack (WHS) : Positif (pas de violation de Hold).

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	3.898	4	30	Counter/coun_g_reg[18]/C	Counter/coun_g_reg[15]/D	4.122	1.014	3.108	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 2	3.907	4	30	Counter/coun_g_reg[18]/C	Counter/coun_g_reg[16]/D	4.150	1.042	3.108	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 3	3.960	4	30	Counter/coun_g_reg[18]/C	Counter/coun_g_reg[22]/D	4.057	1.014	3.043	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 4	3.967	4	30	Counter/coun_g_reg[18]/C	Counter/coun_g_reg[23]/D	4.054	1.014	3.040	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 5	3.975	4	30	Counter/coun_g_reg[18]/C	Counter/coun_g_reg[25]/D	4.083	1.040	3.043	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 6	3.978	4	30	Counter/coun_g_reg[18]/C	Counter/coun_g_reg[24]/D	4.080	1.040	3.040	8.0	sys_clk_pin	sys_clk_pin		0.035
Path 7	3.982	4	30	Counter/coun_g_reg[18]/C	Counter/coun_g_reg[26]/D	4.036	1.014	3.022	8.0	sys_clk_pin	sys_clk_pin		0.035

Bien que le Path 2 présente le délai de propagation physique le plus élevé (4.15 ns), l'analyse temporelle de Vivado montre que le Path 1 possède le Slack le plus faible (3.898ns).

Généralement, le chemin critique est défini par le chemin ayant la marge temporelle (Slack) la plus contraignante. C'est donc le Path 1 qui est le chemin critique de notre design. Néanmoins, avec un Slack positif de près de 3.9 ns sur une période requise de 10.0 ns (horloge à 100 MHz), toutes nos contraintes de setup sont largement respectées.

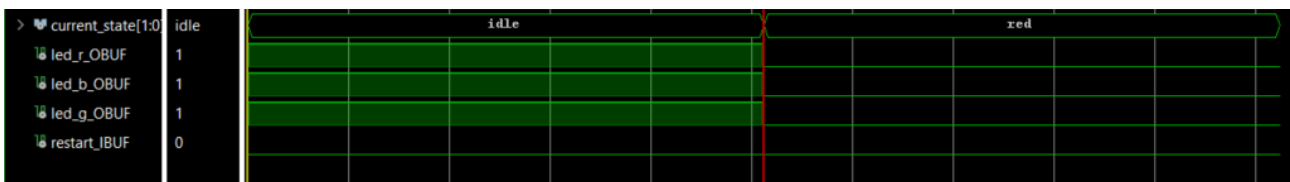


Le chemin critique extrait (visible sur le schéma technologique ci-dessous) correspond au trajet logique le plus long au sein de notre code, situé entre la bascule *counter_reg[15]* à la bascule *counter_reg[18]* de notre module « **counter_unit** ».

Physiquement, ce chemin correspond à la logique de calcul de la retenue et de comparaison de notre compteur de cycles à 27 bits. Pour effectuer l'incrémement (compteur + 1) et valider la condition de remise à zéro à 100 000 000, le signal doit se propager à travers 4 niveaux de portes logiques combinatoires (LUT) entre ces deux registres.

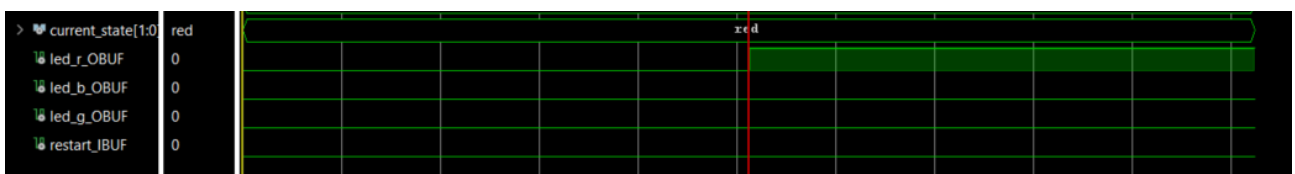
2.12. Question 12

Afin de vérifier le bon fonctionnement sur la carte, une validation via l'outil de debug virtuel (ILA) avec des sondes associées aux différents signaux à observer, ainsi qu'une démonstration vidéo sur carte seront réalisées :

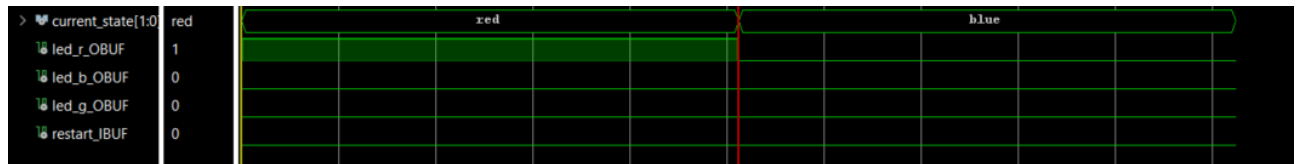


Dans un premier temps, un trigger sur le signal « *current_state* » est placé pour visualiser le changement de valeur et donc d'état à chaque transition. Le trigger est activé pour la valeur '01' donc LED rouge dans le chronogramme ci-dessus.

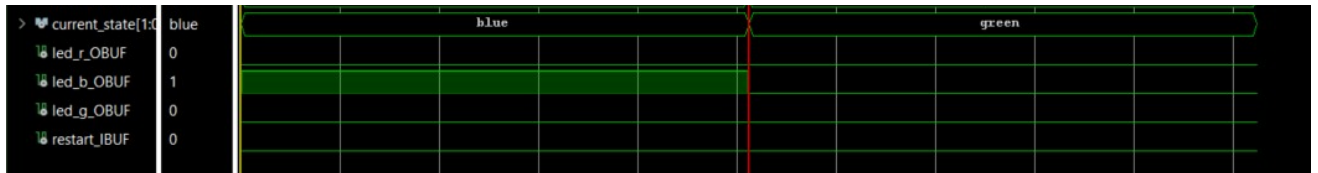
Cette image montre le cycle ON/OFF de la LED.



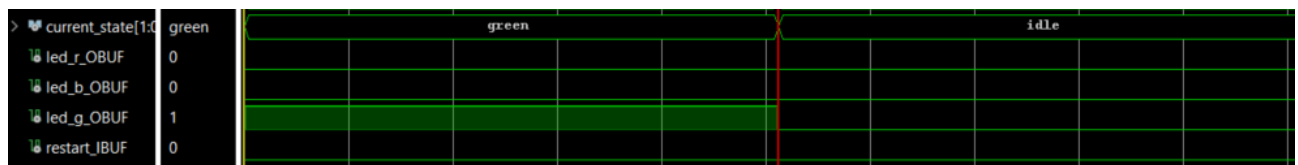
Trigger avec current_state = blue ('10')



Trigger avec current_state = green ('11')



Dernier trigger avec current_state = idle ('00') → retour à l'état initial :



Chaque état respecte bien sa transition. Le fonctionnement nominal est ainsi validé.

Une démonstration vidéo est également disponible pour montrer le bon fonctionnement de l'application de ce TP. (disponible dans le fichier README du dossier TP3 qui dirige vers un lien Google drive).